

AetherNexus Enterprise Orchestrator

Technical Product Manual & Deployment Architecture Specification

Document ID: AN-MAN-2026-V3

Release Date: May 14, 2026

Software Version: v3.1.2-Build892

Classification: Internal Technical Documentation

1. Executive Product Overview

The **AetherNexus Enterprise Orchestrator** is a next-generation local AI model orchestration platform designed specifically for secure enterprise infrastructure. By enabling fully isolated, multi-tenant local inference, AetherNexus eliminates the compliance risks, latency bottlenecks, and structural third-party data dependencies inherent to public cloud-hosted Large Language Models (LLMs).

Engineered to operate natively inside strictly sandboxed homelabs, edge nodes, or decentralized hybrid cloud private networks, the suite orchestrates dense model parameters, embedding pipelines, and neural databases locally. Its main architecture leverages hardware accelerators natively without relying on hyper-scale external compute nodes.

2. System Architecture & Subsystems

AetherNexus v3.1 breaks its workloads into three atomic, resilient subsystems designed to support agentic AI loops and high-concurrency retrieval workflows:

2.1 OmniInference Compute Framework

The OmniInference Engine coordinates dynamic weight mapping and tensor execution across target memory spaces. Built to maximize local hardware, it features custom unified memory allocation models tailored for Apple Silicon (M-series architectures with up to 24GB or greater allocations) and Linux-driven Docker environments running bare-metal NVIDIA runtimes.

- **Adaptive Quantization Matrix (AQM):** Seamlessly handles mixed-precision switching dynamically from FP16 down to 4-bit GGUF files based on instant VRAM contention limits.
- **Context-Stretching Architecture:** Employs proprietary context shifting techniques to maintain performance overheads up to a maximum hard limit of 32,768 contextual tokens.

RAG Performance Note: Context Overhead

When embedding highly granular contextual strings, keeping the token context width within 8,192 tokens preserves processing speeds at an optimal 42 tokens per second (t/s) under standard 24GB memory ceilings.

2.2 ChronosVector Indexing Database

The native vector registry database, code-named ChronosVector, provides low-latency hierarchical navigable small world (HNSW) space indexing. It runs concurrently with inferencing clusters to support real-time generation injection pipelines.

3. Hardware & Software Compatibility Requirements

To ensure stable multi-turn conversational processing loops and prevent execution crashes during dense RAG document ingestions, deployments must meet the following hardware requirements:

Component Specification	Minimum Baseline	Recommended Deployment Standard
Processor Architecture	x86_64 AVX2 Compatible or ARM64	Apple Silicon M2/M3 Max or AMD EPYC 32-Core
Volatile Memory (RAM)	16 GB Unified/System RAM	24 GB or 64 GB Unified Memory Array
Storage Topology	SATA SSD with 500 MB/s read	NVMe M.2 SSD via PCIe Gen4 (4500+ MB/s read)
Virtualization Layer	Docker CE v24.0+ / Ubuntu Server	Proxmox VE v8.1+ containing dedicated Debian Linux VMs

4. Secure Configuration API Examples

System engineers can interface with the orchestration cluster locally using loopback network calls. Below is the standard payload sequence for initializing an internal context chunk synchronization sequence against the ChronosVector engine.

```
POST /api/v3/orchestration/sync-context HTTP/1.1
Host: localhost:8443
Authorization: Bearer AN_SECURE_LOCAL_KEY_77194
Content-Type: application/json

{
  "collection_identity": "secops-knowledge-base",
  "embedding_model_affinity": "local-bge-large-en-v1.5",
  "chunk_size_limit": 512,
  "overlap_ratio": 0.15,
  "enable_gpu_swapping": true
}
```

5. Enterprise Security Compliance & Protocols

The security boundary of AetherNexus is defined by zero external data transmission.

- 1. Local Inbound Protection:** All API interactions run exclusively via secure local endpoints or custom loopback configurations using isolated TLS 1.3 encryption paths.
- 2. Role-Based Telemetry:** No analytical tracking metadata or model outputs are transmitted outside the internal subnet boundary. All model execution telemetry logs remain inside the designated storage volume.
- 3. Exploit Mitigation:** Model operational paths incorporate automatic sanitization routines targeting Prompt Injection threats and recursive Agentic Loop Execution locks.

6. Error Codes and Resolution Matrix

Use this troubleshooting reference guide to configure automatic recovery triggers within orchestration microservices during RAG parsing loops:

Error Code Ident	Root Operational Cause	Automated Resolution Routine
ERR_NEXUS_OOM_102	VRAM Allocation exceeded during a context token expansion window.	Drop precision to 4-bit GGUF parameters or flush context cache registers.
ERR_VECTOR_COLL_404	Target namespace collection identity is missing inside ChronosVector.	Execute a primary schema creation payload via the metadata setup endpoint.
ERR_AUTH_REJECT_403	The provided Bearer authorization string lacks operational access.	Verify local token generation variables inside the private server environment.
ERR_AGENT_RECURSION_550	An AI agentic script has initiated an infinite self-calling execution cycle.	Force terminate execution thread and reset state context window to null.